SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	T. Kiran Kumar	**Reg. No.:**	192472321
Section / Batch:	CSE(AI)	**Date:**	19-08-2026

Code of Conduct

I T. Kiran Kumar *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: T. Kiran Kumar

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- C(machine) = 3
- C(machine learning) = 3
- C(learning improves) = 1
- C(learning enables) = 1
- C(learning drives) = 1
- C(learning improves business) = 1

The interpolation weights are:

- λ_1 = 0.5 (Trigram)
- λ_2 = 0.3 (Bigram)
- λ_3 = 0.2 (Unigram)

The prediction probabilities after the word **"learning"** are:

- P(improves) = 0.33
- P(enables) = 0.33
- P(drives) = 0.33

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **P(learning | machine)**. Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **P(improves) = 0.33, P(enables) = 0.33, P(drives) = 0.33**. Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- P(book | VB) = 0.7
- P(book | NN) = 0.3
- P(Start → VB) = 0.6
- P(Start → NN) = 0.4

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

CASE STUDY 1: E-COMMERCE SMART SEARCH AND PRODUCT PREDICTION SYSTEM

1. Calculate MLE of P(learning | machine)

The MLE formula is:

P(learning | machine) = C(machine learning) / C(machine)

Given:

C(machine learning) = 3

C(machine) = 3

Therefore:

P(learning | machine) = 3/3 = 1.0

This means every occurrence of "machine" is followed by "learning" in the corpus. Hence, the model strongly predicts learning after machine.

2. Apply the Backoff Model

The sequence "machine learning transforms" is unseen.

- Trigram: machine learning transforms → 0
- Bigram: learning transforms → 0
- Unigram: transforms → 0

Therefore, without smoothing:

P(transforms | machine learning) = 0

Backoff is useful because it allows the system to use lower-order N-grams when higher-order sequences are unseen. In practical systems, smoothing or an <UNK> token can give unseen words a small probability.

3. Deleted Interpolation Probability

Formula:

$P = \lambda_1 P(\text{trigram}) + \lambda_2 P(\text{bigram}) + \lambda_3 P(\text{unigram})$

For machine learning improves:

Trigram:

P(improves | machine learning) = 1/3 = 0.3333

Bigram:

P(improves | learning) = 1/3 = 0.3333

Unigram:

P(improves) = 1/18 = 0.0556

Therefore:

P = (0.5 × 0.3333) + (0.3 × 0.3333) + (0.2 × 0.0556)

P = 0.2778

Interpolation improves robustness by combining trigram, bigram, and unigram information.

4. Entropy

Given:

P(improves) = 0.33

P(enables) = 0.33

P(drives) = 0.33

Formula:

$H = -\Sigma P(x)\log_2 P(x)$

Therefore:

$H \approx 1.58$ bits

Since all three words have almost equal probabilities, the entropy is high. This indicates that the system is uncertain about the next word.

5. Python Program

```
import nltk
import numpy as np
import math
from collections import Counter
from nltk.util import ngrams

corpus = [
    "machine learning improves business",
    "machine learning enables automation",
    "machine learning drives innovation"
]

tokens = " ".join(corpus).lower().split()

unigrams = Counter(ngrams(tokens, 1))
bigrams = Counter(ngrams(tokens, 2))
trigrams = Counter(ngrams(tokens, 3))

# MLE
p_mle = bigrams[("machine", "learning")] / unigrams[("machine",)]
print("P(learning | machine) =", p_mle)

# Backoff
if trigrams[("machine", "learning", "transforms")]:
```

```
    p_backoff = 0
elif bigrams[("learning", "transforms")]:
    p_backoff = 0
else:
    p_backoff = 0

print("P(transforms | machine learning) =", p_backoff)

# Deleted Interpolation
p_tri = 1/3
p_bi = 1/3
p_uni = 1/18

p_interpolation = (
    0.5*p_tri + 0.3*p_bi + 0.2*p_uni
)

print("Interpolated Probability =", p_interpolation)

# Entropy
probabilities = np.array([0.33, 0.33, 0.33])

entropy = -sum(
    p * math.log2(p) for p in probabilities
)

print("Entropy =", entropy)
```

Final Results:

MLE = 1.0

Backoff probability = 0

Interpolation probability = 0.2778

Entropy = 1.58 bits

CASE STUDY 2: AI-POWERED HOSPITAL APPOINTMENT CHATBOT

1. Assign Penn Treebank POS Tags

Sentence 1

"Book an appointment with the doctor."

Book/VB an/DT appointment/NN with/IN the/DT doctor/NN

Here, book = VB because it is an action or command.

Sentence 2

"The book contains medical information."

The/DT book/NN contains/VBZ medical/JJ information/NN

Here, book = NN because it refers to a noun.

Thus, the same word can have different POS tags depending on context.

2. HMM Probability Calculation

For VB:

$P(VB, book) = P(Start \rightarrow VB) \times P(book \mid VB)$

$= 0.6 \times 0.7$

$= 0.42$

For NN:

$P(NN, book) = P(Start \rightarrow NN) \times P(book \mid NN)$

$= 0.4 \times 0.3$

$= 0.12$

Therefore:

VB = 0.42

NN = 0.12

The model favors VB because $0.42 > 0.12$.

3. Rule-Based vs HMM Tagging

Rule-Based Tagging

Advantages:

- Simple and interpretable
- Uses grammatical rules

Limitations:

- Requires manually written rules
- Difficult to handle ambiguity

HMM Tagging

Advantages:

- Uses probabilities
- Handles ambiguity better
- Learns patterns from data

Limitations:

- Requires training data
- Can struggle with unseen words

For a healthcare chatbot, HMM tagging is useful for resolving ambiguous words using context and probabilities.

4. Importance of Penn Treebank POS Tags

A standardized POS tagset provides consistent grammatical information.

It supports:

- Intent detection
- Entity identification
- Sentence understanding
- Response generation

For example, identifying book/VB helps the chatbot recognize an appointment-booking action.

5. Python Program

```
import nltk
from nltk.tokenize import word_tokenize

sentence1 = "Book an appointment with the doctor."
sentence2 = "The book contains medical information."

tokens1 = word_tokenize(sentence1)
tokens2 = word_tokenize(sentence2)

print("Sentence 1 Tokens:", tokens1)
print("Sentence 2 Tokens:", tokens2)

# HMM probabilities
prob_vb = 0.6 * 0.7
prob_nn = 0.4 * 0.3

print("P(VB, book) =", prob_vb)
print("P(NN, book) =", prob_nn)

if prob_vb > prob_nn:
    print("Most likely tag: VB")
else:
    print("Most likely tag: NN")

print("\nPOS Tagged Sentences:")
```

print("Book/VB an/DT appointment/NN with/IN the/DT doctor/NN")

print("The/DT book/NN contains/VBZ medical/JJ information/NN")

Final Results:

VB probability = 0.42

NN probability = 0.12

Most probable tag = VB

CASE STUDY 3: FINANCIAL NEWS POS TAG CORRECTION AND UNCERTAINTY ANALYSIS

1. Apply the transformation rule

Initial tags:

market/NN growth/NN drives/NNS investment/NN

Rule:

Change NNS to VBZ if the preceding word is NN.

Since growth/NN precedes drives/NNS, the rule changes drives to VBZ.

Corrected sequence:

market/NN growth/NN drives/VBZ investment/NN

2. Analyze the error in "drives"

In "Market growth drives investment", "drives" is the main verb. The subject market growth is singular, so the correct tag is VBZ.

The surrounding grammatical context helps identify it as a verb rather than a plural noun.

3. Calculate word probabilities

Total frequency:

500 + 350 + 180 + 420 = 1450

Word	Frequency	Probability
market	500	0.3448
growth	350	0.2414
drives	180	0.1241
investment	420	0.2897

Corpus frequencies help statistical NLP systems estimate word likelihoods and improve tagging and prediction.

4. Calculate entropy

Formula:

$H = -\Sigma P(x)\log_2 P(x)$

Using the four probabilities:

$H \approx 1.916$ bits

The transformation only changes the POS tag of "drives"; it does not change word frequencies.

Therefore:

Entropy before transformation = 1.916 bits

Entropy after transformation = 1.916 bits

Thus, POS accuracy improves, but the word-frequency entropy remains unchanged.

5. Python Program

```
import nltk
import math
from nltk.tokenize import word_tokenize

sentence = "Market growth drives investment."

tokens = word_tokenize(sentence)

# Initial tags
initial_tags = [
    ("market", "NN"),
    ("growth", "NN"),
    ("drives", "NNS"),
    ("investment", "NN")
]

print("Initial Tags:")
print(initial_tags)

# Transformation rule
corrected_tags = initial_tags.copy()

for i in range(1, len(corrected_tags)):
    word, tag = corrected_tags[i]
    previous_word, previous_tag = corrected_tags[i-1]

    if tag == "NNS" and previous_tag == "NN":
        corrected_tags[i] = (word, "VBZ")

print("\nCorrected Tags:")
```

```
print(corrected_tags)

# Word frequencies
frequencies = {
    "market": 500,
    "growth": 350,
    "drives": 180,
    "investment": 420
}

total = sum(frequencies.values())

probabilities = {
    word: freq / total
    for word, freq in frequencies.items()
}

print("\nWord Probabilities:")
for word, probability in probabilities.items():
    print(word, "=", round(probability, 4))

# Entropy
entropy = -sum(
    p * math.log2(p)
    for p in probabilities.values()
)

print("\nEntropy =", round(entropy, 4), "bits")

print("\nFinal POS Tags:")
for word, tag in corrected_tags:
    print(word + "/" + tag)
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	4	4	4	3	3	20	92
2	4	4	5	5	4	25	
3	5	4	5	4	5	20	

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____